

bound $RL/\sqrt{T}$ on the accuracy of the solution after T iterations in Theorem 33.11 becomes $O(B)L/\sqrt{T} = O(B^2/\sqrt{T})$.

Exercises

33.3-1

Prove Lemma 33.6. Start from the definition of a convex function given in equation (33.18). (*Hint:* You can prove the statement when $n = 1$ first. The proof for general values of n is similar.)

33.3-2

Prove Lemma 33.7.

33.3-3

Prove equation (33.29). (*Hint:* The proof for $n = 1$ dimension is straightforward. The proof for general values of n dimensions follows along similar lines.)

33.3-4

Show that the function f in equation (33.32) is a convex function of the variables $w_0, w_1, \dots, w_n$.

33.3-5

Compute the gradient of expression (33.33) and explain how to evaluate the gradient in $O(nm)$ time.

33.3-6

Consider the function f defined in equation (33.32), and suppose that you have a bound $\|\mathbf{w}\| \leq B$, as is considered in the discussion on regularization. Show that $L = O(B)$ in this case.

33.3-7

Equation (33.2) on page 1009 gives a function that, when minimized, gives an optimal solution to the k -means problem. Explain how to use gradient descent to solve the k -means problem.

Problems

33-1 Newton's method

Gradient descent iteratively moves closer to a desired value (the minimum) of a function. Another algorithm in this spirit is known as *Newton's method*, which is an iterative algorithm that finds the root of a function. Here, we consider Newton's method which, given a function $f: \mathbb{R} \rightarrow \mathbb{R}$, finds a value x^* such that $f(x^*) = 0$. The algorithm moves through a series of points $x^{(0)}, x^{(1)}, \dots$. If the algorithm is currently at a point $x^{(t)}$, then to find point $x^{(t+1)}$, it first takes the equation of the line tangent to the curve at $x = x^{(t)}$,

$$y = f'(x^{(t)})(x - x^{(t)}) + f(x^{(t)}).$$

It then uses the x -intercept of this line as the next point $x^{(t+1)}$.

a. Show that the algorithm described above can be summarized by the update rule

$$x^{(t+1)} = x^{(t)} - \frac{f(x^{(t)})}{f'(x^{(t)})}.$$

We restrict our attention to some domain I and assume that $f'(x) \neq 0$ for all $x \in I$ and that $f''(x)$ is continuous. We also assume that the starting point $x^{(0)}$ is sufficiently close to x^* , where “sufficiently close” means that we can use only the first two terms of the Taylor expansion of $f(x^*)$ about $x^{(0)}$, namely

$$f(x^*) = f(x^{(0)}) + f'(x^{(0)})(x^* - x^{(0)}) + \frac{1}{2}f''(\gamma^{(0)})(x^* - x^{(0)})^2, \quad (33.34)$$

where $\gamma^{(0)}$ is some value between $x^{(0)}$ and x^* . If the approximation in equation (33.34) holds for $x^{(0)}$, it also holds for any point closer to x^* .

b. Assume that the function f has exactly one point x^* for which $f(x^*) = 0$. Let $\epsilon^{(t)} = |x^{(t)} - x^*|$. Using the Taylor expansion in equation (33.34), show that

$$\epsilon^{(t+1)} = \frac{|f''(\gamma^{(t)})|}{2|f'(\gamma^{(t)})|} \epsilon^{(t)},$$

where $\gamma^{(t)}$ is some value between $x^{(t)}$ and x^* .

c. If

$$\frac{|f''(\gamma^{(t)})|}{2|f'(\gamma^{(t)})|} \leq c$$

for some constant c and $\epsilon^{(0)} < 1$, then we say that the function f has **quadratic convergence**, since the error decreases quadratically. Assuming that f has quadratic convergence, how many iterations are needed to find a root of $f(x)$ to an accuracy of δ ? Your answer should include δ .

d. Suppose you wish to find a root of the function $f(x) = (x - 3)^2$, which is also the minimizer, and you start at $x^{(0)} = 3.5$. Compare the number of iterations needed by gradient descent to find the minimizer and Newton's method to find the root.

33-2 *Hedge*

Another variant in the multiplicative-weights framework is known as HEDGE. It differs from WEIGHTED MAJORITY in two ways. First, HEDGE makes the prediction randomly—in iteration t , it assigns a probability $p_i^{(t)} = w_i^{(t)} / Z^{(t)}$ to expert E_i , where $Z^{(t)} = \sum_{i=1}^n w_i^{(t)}$. It then chooses an expert E_{j^*} according to this probability distribution and predicts according to E_{j^*} . Second, the update rule is different. If an expert makes a mistake, line 16 updates that expert's weight by the rule $w_i^{(t+1)} = w_i^{(t)} e^{-\epsilon}$, for some $0 < \epsilon < 1$. Show that the expected number of mistakes made by HEDGE, running for T rounds, is at most $m^* + (\ln n)/\epsilon + \epsilon T$.

33-3 *Nonoptimality of Lloyd's procedure in one dimension*

Give an example to show that even in one dimension, Lloyd's procedure for finding clusters does not always return an optimum result. That is, Lloyd's procedure may terminate and return as a result a set C of

clusters that does not minimize $f(S, C)$, even when S is a set of points on a line.

33-4 *Stochastic gradient descent*

Consider the problem described in Section 33.3 of fitting a line $f(x) = ax + b$ to a given set of point/value pairs $S = \{(x_1, y_1), \dots, (x_T, y_T)\}$ by optimizing the choice of the parameters a and b using gradient descent to find a best least-squares fit. Here we consider the case where x is a real-valued variable, rather than a vector.

Suppose that you are not given the point/value pairs in S all at once, but only one at a time in an online manner. Furthermore, the points are given in random order. That is, you know that there are n points, but in iteration t you are given only (x_i, y_i) where i is independently and randomly chosen from $\{1, \dots, T\}$.

You can use gradient descent to compute an estimate to the function. As each point (x_i, y_i) is considered, you can update the current values of a and b by taking the derivative with respect to a and b of the term of the objective function depending on (x_i, y_i) . Doing so gives you a stochastic estimate of the gradient, and you can then take a small step in the opposite direction.

Give pseudocode to implement this variant of gradient descent. What would the expected value of the error be as a function of T , L , and R ? (*Hint*: Replicate the analysis of GRADIENT-DESCENT in Section 33.3 for this variant.)

This procedure and its variants are known as *stochastic gradient descent*.

Chapter notes

For a general introduction to artificial intelligence, we recommend Russell and Norvig [391]. For a general introduction to machine learning, we recommend Murphy [340].

Lloyd's procedure for the k -means problem was first proposed by Lloyd [304] and also later by Forgy [151]. It is sometimes called "Lloyd's

algorithm” or the “Lloyd-Forgy algorithm.” Although Mahajan et al. [310] showed that finding an optimal clustering is NP-hard, even in the plane, Kanungo et al. [241] have shown that there is an approximation algorithm for the k -means problem with approximation ratio $9 + \epsilon$, for any $\epsilon > 0$.

The multiplicative-weights method is surveyed by Arora, Hazan, and Kale [25]. The main idea of updating weights based on feedback has been rediscovered many times. One early use is in game theory, where Brown defined “Fictitious Play” [74] and conjectured its convergence to the value of a zero-sum game. The convergence properties were established by Robinson [382].

In machine learning, the first use of multiplicative weights was by Littlestone in the Winnow algorithm [300], which was later extended by Littlestone and Warmuth to the weighted-majority algorithm described in Section 33.2 [301]. This work is closely connected to the boosting algorithm, originally due to Freund and Shapire [159]. The multiplicative-weights idea is also closely related to several more general optimization algorithms, including the perceptron algorithm [328] and algorithms for optimization problems such as packing linear programs [177, 359].

The treatment of gradient descent in this chapter draws heavily on the unpublished manuscript of Bansal and Gupta [35]. They emphasize the idea of using a potential function and using ideas from amortized analysis to explain gradient descent. Other presentations and analyses of gradient descent include works by Bubeck [75], Boyd and Vanderberghe [69], and Nesterov [343].

Gradient descent is known to converge faster when functions obey stronger properties than general convexity. For example, a function f is *α -strongly convex* if $f(\mathbf{y}) \geq f(\mathbf{x}) + \langle (\nabla f)(\mathbf{x}), (\mathbf{y} - \mathbf{x}) \rangle + \alpha \|\mathbf{y} - \mathbf{x}\|^2$ for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$. In this case, GRADIENT-DESCENT can use a variable step size and return $\mathbf{x}^{(T)}$. The step size at step t becomes $\gamma_t = 1/(\alpha(t + 1))$, and the procedure returns a point such that $f(\mathbf{x}\text{-avg}) - f(\mathbf{x}^*) \leq L^2/(\alpha(T + 1))$. This convergence is better than that of Theorem 33.8 because the number of iterations needed is linear, rather than quadratic, in the

desired error parameter ϵ , and because the performance is independent of the initial point.

Another case in which gradient descent can be shown to perform better than the analysis in Section 33.3 suggests is for smooth convex functions. We say that a function is *β -smooth* if $f(y) \leq f(x) + \langle (\nabla f)(x), (y - x) \rangle + \frac{\beta}{2} \|y - x\|^2$. This inequality goes in the opposite direction from the one for μ -strong convexity. Better bounds on gradient descent are possible here as well.

¹ Although the curve in Figure 33.4 looks concave, according to the definition of convexity that we'll see below, the function f in the figure is convex.

² A function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is closed if, for each $\alpha \in \mathbb{R}$, the set $\{x \in \text{dom}(f) : f(x) \leq \alpha\}$ is closed, where $\text{dom}(f)$ is the domain of f .